

THE UNIVERSITY OF AZAD JAMMU & KASHMIR, MUZAFFARABAD



COURSE TITLE	SDA
COURSE CODE	SE-2102
TITLE	LAB # 10
STUDENT NAME	HASEEB SHEIKH
STUDENT ROLL NUMBER	2024-SE-31
INSTRUCTOR NAME	ENGR. DANIYAL NAZIR
SUBMISSION DATE	JAN 31, 2026

DEPARTMENT OF SOFTWARE ENGINEERING

SD&A

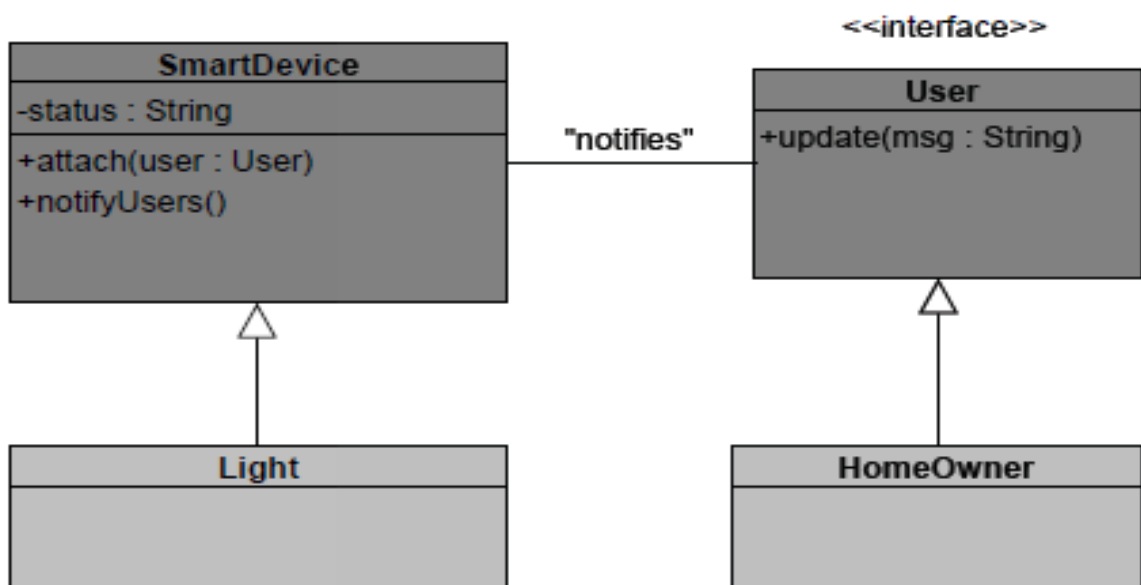
Lab 10

Analysis

Analysis:

1. **Creational Pattern:** Factory Method.
Reason: It allows the creation of different device objects (Lights, Fans) dynamically without specifying their exact classes.
2. **Behavioral Pattern:** Observer Pattern.
Reason: It is perfect for notifying multiple users (subscribers) automatically whenever a device's status changes.
3. **Structural Pattern:** Facade Pattern.
Reason: It provides a simplified, unified interface to control complex subsystems of devices.

Diagram



Observer Pattern Implementation (C++)

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5  // The Observer Interface
6  class User {
7  public:
8      virtual void update(string message) = 0;
9  };
10 // The Subject (Base Class)
11 class SmartDevice {
12     vector<User*> observers;
13 public:
14     void attach(User* user) {
15         observers.push_back(user);
16     }
17     void notifyUsers(string message) {
18         for (User* user : observers) {
19             user->update(message);
20         }
21     }
22 };
23 // Concrete Observer
24 class HomeOwner : public User {
25     string name;
26 public:
27     HomeOwner(string n) : name(n) {}
28     void update(string message) override {
29         cout << "Notification for " << name << ": " << message << endl;
30     }
31 };
```